If you access the code for this chapter on this book's GitHub at http://mng.bz/JzKp, you will find the code we used to record frames of the live game play during training. We also recorded the real-time changes in the action-value distributions so you can see how the game play affects the predicted distributions and vice versa. We do not include that code here in the book, as it would take too much space. In listing 7.13 we initialize the hyperparameters and variables we'll need for our Dist-DQN algorithm.

Listing 7.13. Dist-DQN plays Freeway, preliminaries

```
import gym
from collections import deque
env = gym.make('Freeway-ram-v0')
aspace = 3
env.env.get_action_meanings()

vmin,vmax = -10,10
replay_size = 200
batch_size = 50
nsup = 51
dz = (vmax - vmin) / (nsup-1)
support = torch.linspace(vmin,vmax,nsup)

replay = deque(maxlen=replay_size)            1
lr = 0.0001                                   2
gamma = 0.1                                   3
epochs = 1300
eps = 0.20                                    4
eps_min = 0.05                                5
priority_level = 5                            6
update_freq = 25                              7

#Initialize DQN parameter vector
tot_params = 128*100 + 25*100 + aspace*25*51  8
theta = torch.randn(tot_params)/10.           9
theta.requires_grad=True
theta_2 = theta.detach().clone()              10

losses = []
cum_rewards = []                              11
renders = []
state = preproc_state(env.reset())
```

- ***1*** Experience replay buffer using the deque data structure
- ***2*** Learning rate
- ***3*** Discount factor